GitHub Copilot Instructions: Chit Fund Manager App

1. Project Context & Role

Role: Expert Python Streamlit Developer with a focus on Financial Applications and Data Integrity. **Goal:** Build a robust, multi-stage Streamlit application for managing Chit Funds. The app acts as a UI wrapper around an existing core logic package (`chit_fund_analyzer`) and uses Google Sheets as a database with ACID-like consistency checks.

2. Technical Stack

- **Frontend:** Streamlit, Streamlit AgGrid (for editable tables), Plotly (for charts).
- **Core Logic:** `chit_fund_analyzer` (Local package - assume it is already installed/available in path).
- **Database:** Google Sheets via `gspread` and `google-auth-oauthlib` .
- **State Management:** `st.session_state` .
- **Styling:** Custom CSS for a professional financial dashboard look.

3. Directory Structure

Ensure the code is generated following this modular structure:

```
project root/
├── chit fund analyzer/    # (Existing Package - DO NOT MODIFY)
│   ├──   init  .py
│   ├── models.py
│   ├── analyzer.py
│   ├── ...
├── streamlit app/         # (Target Directory for App)
│   ├──   init  .py
│   ├── main.py            # Entry point
│   ├── auth.py            # OAuth & Session handling
│   ├── db.py              # Google Sheets DAL with Optimistic Locking
│   ├── utils.py           # UI helpers
│   └── stages/
│       ├──   init  .py
│       ├── login.py       # Stage 0
│       ├── dashboard.py   # Stage 1 (Selection/Creation/Edit)
│       ├── installments.py# Stage 2 (Tracking)
│       └── analytics.py   # Stage 3 (Scenarios)
├── .streamlit/
│   └── secrets.toml       # API Credentials
└── requirements.txt
```

4. Implementation Details by Module

A. Data Access Layer (`streamlit_app/db.py`)

Critical Requirement: Implement **Optimistic Locking** to ensure data integrity in Google Sheets.

1. **Class** `ChitFundDB` :
   - Initialize with `gspread.Client` .
   - **Schema Enforcement**: Ensure tabs `chits` and `installments` exist with correct headers.
2. **Concurrency Logic (The "ACID" Check)**:
   - Every row in the `chits` sheet must have a `version` column (integer).
   - **Read Operation**: Fetch data and keep the `version` number in memory.
   - **Write Operation**: Before updating a row, fetch the *current* version from the sheet.
     - `if current_version == read_version` : Proceed with update and increment version to `v + 1` .
     - `else` : Raise `StaleDataError` and prompt user to refresh.
3. **Methods**:
   - `get_all_chits()` : Returns list of dicts.
   - `create_chit(metadata: dict)` : atomic append to `chits` + bulk append to `installments` .
   - `update_chit_metadata(chit_id, metadata: dict)` : Update name, description, etc.
   - `get_installments(chit_id)` : Join logic if necessary, or simple filter.
   - `update_installments(chit_id, updates: List[dict])` : Batch update using cell ranges.

B. Authentication (`streamlit_app/auth.py`)

1. Use `google_auth_oauthlib` .
2. Check for `st.secrets["google_oauth"]` first. Fallback to `client_secret.json` for local dev.
3. Store the `credentials` object in `st.session_state['credentials']` .
4. **Persistence**: Do not over-engineer cookie management initially. Rely on Streamlit's session persistence. If a token is expired, auto-refresh using the refresh token.

C. Stage Management (`streamlit_app/main.py`)

1. Use a robust state machine pattern.
2. `st.session_state['current_stage']` determines which module from `stages/` is rendered.
3. **Navigation Sidebar**: Only show stages accessible based on current context (e.g., don't show "Installments" if no Chit is selected).

D. Stages

Stage 1: Dashboard (`stages/dashboard.py`)

- Tab 1: Select / Edit Existing Chit:
  - Dropdown to select a Chit.
  - **Edit Mode**: Provide an "Edit Details" button that opens a form to update the Chit's Name, Description, or other metadata (excluding immutable fields like ID). Save changes via `db.update_chit_metadata` .
  - Selection loads data into `st.session_state['selected_chit']` and advances stage.
- Tab 2: Create New Chit:
  - **Inputs**: Restrict inputs to **only** the essentials required for `ChitFundConfig` :
    - `Chit Name`
    - `Total Installments` (int)
    - `Full Chit Value` (decimal)
    - `Chit Frequency` (per year)
    - `First Installment Date`
  - **Note**: Do **not** ask for `previous_installments` here. A new chit implies 0 previous installments.
  - **Action**: "Initialize Chit" button -> calls `db.create_chit` .

Stage 2: Installment Tracking (`stages/installments.py`)

- **Gatekeeper**: If `start_date > today` , show a warning and offer to skip to Analysis.
- **Data Editor**: Use `st.data_editor` (or AgGrid) for the installments table.
- Business Logic & Reactive Formulas:
  - **Field**: `Amount Paid` (by a single non-winner candidate).
  - **Reactive Trigger**: When the user updates the `Amount Paid` or `Discount` , the system must immediately recalculate dependent fields.
  - **Formula**: $$ \text{Amount Paid} = \frac{\text{Total Value} - \text{Discount}}{\text{Total Installments} - \text{Current Installment Number}} $$
  - **Implementation**: Ensure this formula is applied to auto-calculate/validate the `Amount Paid` field.
- Analyzer Integration:
  - On every data change, reconstruct the `ChitFundConfig` object.
  - Call `chit_fund_analyzer.analyzer.ChitFundAnalyzer.analyze()` to get the latest `Prize Amount` and `IRR` .
  - Display these metrics in the UI (KPI Cards) immediately.
- Validation:

    - Enforce: `Amount Paid` cannot be < 60% of winner installment (use `ChitFundConfig` validation logic).
- **Save**: Commit changes to DB using `db.update_installments`.

Stage 3: Analysis (`stages/analytics.py`)

- Use `chit_fund_analyzer.scenario.ScenarioAnalyzer`.
- **Inputs**: Sliders for Bid Range, Number of Scenarios.
- **Outputs**:
    - Line Chart: Bid Amount vs Annual IRR.
    - Table: Detailed Cashflow breakdown.
    - Export: "Download Report" button (Excel).

5. Coding Standards & Best Practices

1. **Type Hinting**: Use `typing.List`, `typing.Dict`, `typing.Optional` everywhere.
2. **Error Handling**: Wrap all Google API calls in `try/except` blocks. Show user-friendly `st.error` messages, not stack traces.
3. **Docstrings**: Google-style docstrings for all functions.
4. **Configuration**: Use `pydantic` models for UI form validation before sending data to the DB.
5. **Performance**: Cache DB reads using `@st.cache_data(ttl=60)`. Clear cache on successful writes.

6. Reference: `chit_fund_analyzer` Interface

Assume these classes exist and import them as needed:

```
from chit_fund_analyzer.models import ChitFundConfig, ChitFundAnalysisResult
from chit_fund_analyzer.analyzer import ChitFundAnalyzer
from chit_fund_analyzer.scenario import ScenarioAnalyzer

# Example Usage
config = ChitFundConfig(...)
analyzer = ChitFundAnalyzer(config)
result = analyzer.analyze() # returns ChitFundAnalysisResult
```